

Cours 6

Architecture IoT, Cloud Robotique et Architectures Événementielles

Sujets du cours :

- Architecture IoT (couches, sécurité, protocoles)
- Cloud Robotique : modèle client-serveur
- Edge/Cloud continuum
- Architecture EDA réactive (événementielle)
- Monolithique vs. microservices

Table des matières

1	Architecture IoT	2
1.1	Les 4 couches de l'IoT	2
1.2	Sécurité : menaces et protections	2
1.3	Comparaison des protocoles IoT	2
1.4	Sujets de recherche liés	2
2	Architecture Client-Serveur et Cloud Robotique	3
2.1	Modèle client-serveur en 3 couches	3
2.2	API, requêtes et indexation	3
2.3	Cloud Robotique	3
3	Edge/Cloud Continuum	3
3.1	Répartition Edge vs Cloud	4
3.2	Organisation Edge vs Cloud	4
4	Architecture Événementielle (EDA)	4
4.1	Caractéristiques principales	4
4.2	Patron de conception Publish/Subscribe	4
4.3	Tolérance aux pannes avec EDA	5
5	Microservices vs Monolithique	5
6	Conclusion	5

Architecture IoT

• Les 4 couches de l'IoT

Une architecture IoT se décompose en 4 couches, chacune associée à un domaine d'ingénierie :

Couche	Nom	Domaine	Contraintes
4	Business	Développement logiciel	Besoins hybrides
3	Processing	Data engineering	Montée en charge (scaling)
2	Transport	Réseaux	Chiffrement
1	Perception	Électronique	Réglementations

La **sécurité** est une couche transversale qui couvre toutes les autres.

• Sécurité : menaces et protections

Menace	Protection associée
Phishing, Malware	Stabilité réseau
Sniffing	Chiffrement (HTTPS, MQTT)
Déni de service (DoS)	Politique de relance (retry)
DNS Tunneling	Scalabilité
Spoofing, Man-in-the-middle	Authentification, TLS
Injection de code	Validation des entrées
Supply Chain Attacks	Audit des dépendances

• Comparaison des protocoles IoT

Protection	HTTP	HTTPS	Kafka	MQTT
Stabilité réseau	✓	✓	×	✓
Anti-sniffing	×	✓	✓	✓
Retry policy	×	×	✓	✓
Scalabilité	×	×	✓	×
Anti-MitM	×	×	~	✓

À retenir

MQTT est le protocole le mieux adapté à l'IoT embarqué : léger, fiable, avec QoS intégré. **Kafka** excelle pour le traitement de données à grande échelle (streaming, data lineage).

• Sujets de recherche liés

- **MQTT-S** : extension sécurisée de MQTT (logique floue contre les attaques DoS, Haripriya & Kulothungan, 2019)
- **Data lineage MQTT/RFID** : traçabilité des données dans la logistique (Arif et al., 2019)
- **Kafka** : data lineage scalable pour flux continus (Cakir et al., 2022)
- **Time Series Databases (TDB)** : stockage et analyse de séries temporelles (Esling & Agon, 2012)

Architecture Client-Serveur et Cloud Robotique

• Modèle client-serveur en 3 couches

Définition : Modèle client-serveur (n-tiers)

Architecture distribuée composée de 3 niveaux :

1. **Clients** : nombreux, envoient des requêtes HTTP
2. **Serveurs intermédiaires** : contrôleurs (MVC), brokers, load balancers, gateways...
3. **Serveurs de données** : peu nombreux (on-premise) ou multiples (cloud)

• API, requêtes et indexation

Exemple

<http://www.gutenberg.org/ebooks/> → liste de livres

<http://www.gutenberg.org/ebooks/49345> → détails du livre n°49345

Problème : les IDs numériques ne sont pas parlants pour un humain.

Solution : un serveur intermédiaire (ex. *Sargon*) fait l'**indexation** :

- Pré-calcul de réponses (mots-clés, noms...)
- Chaque réponse indexée renvoie vers le document original
- Compromis : explosion de l'espace de stockage vs mise à jour automatique

• Cloud Robotique

Définition : Cloud Robotique

Application du modèle client-serveur où **les robots jouent le rôle de clients** :

- Envient des données capteurs au serveur
- Évitent de traiter localement les données
- Reçoivent et exécutent des instructions (actionneurs)

Cycle : **LCM** (Look – Compute – Move), en déportant le Compute au serveur.

Analogie : robot $\approx$ petit cerveau ; serveur $\approx$ supercalculateur.

Attention : le modèle cloud-only est valable **uniquement pour les décisions non critiques** (latence trop élevée pour l'urgence).

Edge/Cloud Continuum

• Répartition Edge vs Cloud

	Edge (robot local)	Cloud (datacenter distant)
Rôle	Actions critiques, faible latence	Optimisation, ML, recherche distribuée
Analogie	Réflexes	Neurones
Exemples	E-STOP, stabilisation drone, prétraitement capteurs, odométrie	Apprentissage, fouille de données, bases distribuées

• Organisation Edge vs Cloud

À retenir

- **Sans edge** $\approx$ algorithmes séquentiels : entrées/sorties simples, non interruptibles
- **Avec edge** $\approx$ algorithmes distribués : synchrone/asynchrone, concurrence, gestion d'interruptions $\Rightarrow$ **programmation réactive (EDA)**

Architecture Événementielle (EDA)

Définition : EDA – Event-Driven Architecture

Architecture où les composants communiquent via des **événements** plutôt que par des appels directs.

• Caractéristiques principales

Concept	EDA réactif	Boucle procédurale (polling)
Mode	Réactif, déclenché par événement	Actif, vérifie en continu
Ressources	CPU peut dormir $\Rightarrow$ économie batterie	CPU toujours actif
Bande passante	Économisée (pas de polling inutile)	Gaspillée
Latence	Milliseconde (tâches critiques)	Dépend de la fréquence de poll

• Patron de conception Publish/Subscribe

Définition : Publish/Subscribe (Pub/Sub)

- Les **producteurs (publishers)** publient des messages sur un **bus d'événements**
- Les **consommateurs (subscribers)** s'abonnent aux topics qui les intéressent
- Les composants sont **découplés** : ils ne se connaissent pas directement

Exemples d'outils EDA : ROS, Mosquitto, Kafka, Pulsar, Azure Event Hubs...

• Tolérance aux pannes avec EDA

L'EDA facilite la tolérance aux pannes grâce au découplage :

- Un nœud défaillant n'arrête pas le reste du système
- Les messages peuvent être mis en file d'attente (persistance)
- Reprise automatique après panne possible (retry, replay)

Microservices vs Monolithique

	Architecture monolithique	Architecture microservices
Structure	Une seule application tout-en-un	Processus indépendants (nœuds, modules)
Déploiement	Simple mais rigide	Flexible, déploiement partiel
Tolérance	Faible : une panne = arrêt total	Élevée : pannes isolées
Complexité	Faible au départ	Plus complexe à orchestrer
Exemples	Application LAMP classique	Docker + Kubernetes, ROS nodes

À retenir

Les microservices correspondent directement au modèle EDA/Pub-Sub : chaque service est un nœud indépendant qui publie et consomme des événements. Cela est fondamental dans la robotique cloud (ROS) et les systèmes distribués en général.

Conclusion

Synthèse du cours 6

- **IoT** : 4 couches (Perception, Transport, Processing, Business) + sécurité transversale. Protocoles : MQTT $\gg$ HTTP pour l'embarqué ; Kafka pour le scaling.
- **Cloud Robotique** : modèle client-serveur où le robot délègue le calcul au cloud, sauf pour les décisions critiques (edge).
- **Edge/Cloud continuum** : réflexes sur le robot, apprentissage dans le cloud.
- **EDA (réactif)** : meilleur que le polling pour les systèmes temps réel (économie de bande passante et de batterie). Patron Pub/Sub.
- **Microservices** : processus indépendants, tolérance aux pannes, orchestration par événements.

Références : Tabbane 2019 ; Haripriya & Kulothungan 2019 ; Arif et al. 2019 ; Cakir et al. 2022 ; Esling & Agon 2012.